SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA3711 – Software Testing

ASSIGNMENT 1 – CO1

Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application

CO1: Testing as an engineering activity, role of process in software quality, testing as a process, basic definitions, software testing principles, tester's role, origins of defects, defect classes, defect repository, defect examples.

Student Name	Mugesh S	**Register Number**	192411055
Department / Section	B.E. - CSE / SLOT A	**Academic Year / Semester**	2024 / MS2 Sep-Dec 2026 (UG)
Faculty Name	Dr. R. Kesavan	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	01/11/2026	**Date of Evaluation**	

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident

I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.
I8	A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital's SMS provider changes its API key.
I10	The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.

Defect data for the last release (100 defects):

Phase	**Defects introduced**	**Defects detected**	**Cost to fix per defect (₹)**	**Remarks**

Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.

Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate Testing as an Engineering Activity **5 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity.

Did the MediQueue team engineer quality into the product, or did they only hope for it?

Discuss:

- The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)

- Which of these characteristics were missing in the MediQueue project, with evidence from the case
- How the absence of each characteristic contributed to the incidents reported
- What an engineered testing approach would have looked like for this project

You are expected to judge the project against engineering practice, not merely define the term.

Evidence Expected

A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

TASK 2 – Evaluate the Role of Process in Software Quality **2 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem.

Is quality mainly a result of good people, or of a good process?

Discuss:

- The role of a defined process in achieving software quality
- Whether better developers alone would have prevented the incidents
- How a process maturity view (for example, TMM levels) would place the current team

Your position must be defended with evidence from the case.

Evidence Expected

A justified position on the role of process in quality, supported by the case data.

TASK 3 – Evaluate Testing as a Process **6 Marks**

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release.

What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Discuss:

- Testing as a process with planning, design, execution, evaluation and reporting activities
- The relationship between verification and validation activities and each development phase
- The phase in which each type of MediQueue defect should have been detected
- Using the defect data table, the cost the hospital paid because defects were found late
- The likely saving if requirements and design reviews had been conducted
- The limits of your estimate and the assumptions it depends on

You must show your calculations and interpret them. Stating that “early testing is cheaper” will not receive full credit.

Evidence Expected

A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

TASK 4 – Apply and Critique Basic Definitions **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The team uses the words “bug”, “error” and “failure” interchangeably in its reports.

Does careless terminology actually harm quality, or is it only a matter of words?

Discuss:

- The meaning of error, fault (defect), failure, test case, test oracle and test bed in the MediQueue context
- For incidents I3, I5 and I6, the error, the fault and the failure

- Why the confusion in terminology made incident I8 possible
- Whether the team needs a common glossary, and what it must contain

Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.

Evidence Expected

Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

TASK 5 – Evaluate the Application of Testing Principles **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The developers claim that "we tested everything and found no errors, so the software was correct".

Which testing principles prove this claim wrong?

Discuss:

- At least four software testing principles that apply to the MediQueue situation
- For each principle, the incident or decision in the case that violates or confirms it
- Which principle, if followed, would have had the largest effect on quality, and why

Do not simply list principles. You are expected to relate each principle to evidence.

Evidence Expected

A justified evaluation of the developers' claim using testing principles linked to evidence from the case.

TASK 6 – Evaluate the Tester's Role **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester.

Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Discuss:

- The role and responsibilities of a tester across the life cycle
- The advantages and limitations of developers testing their own code
- Which incidents a tester would most likely have caught, and at which stage
- The qualities a tester for a healthcare application must have

Your answer must take a position and defend it.

Evidence Expected

A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

TASK 7 – Trace the Origins of Defects **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated.

Where in the life cycle are MediQueue's defects really coming from?

Discuss:

- The origin of each incident: requirements, design, code, testing, documentation or environment
- The evidence in the case that supports each origin you assign
- Incidents that may have more than one origin, and how you decided
- What the overall pattern of origins tells the hospital about where to invest effort

Where an incident could belong to more than one origin, argue for your choice.

Evidence Expected

A justified origin for each incident and an interpretation of the overall pattern of defect origins.

TASK 8 – Evaluate Defect Classes and Severity **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects.

Which classes of defects are most harmful to patients, and do the team's priorities reflect that?

Discuss:

- A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)
- The class, severity and priority of each incident, with reasons
- Why severity and priority may differ for the same defect (for example, I7 and I1)
- Which defect class the team should target first, and why

Simply placing defects in a table is not sufficient. You are expected to justify each classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

TASK 9 – Design and Appraise a Defect Repository **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet.

What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Discuss:

- The fields a defect repository for MediQueue must contain, and why each is needed
- How the repository supports defect analysis, process improvement and prevention
- The weaknesses of the current e-mail and spreadsheet approach

You are expected to justify your repository design, not just list fields.

Evidence Expected

A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes.

For each selected defect, what single change to the process would have prevented it?

Discuss:

- The root cause of each selected defect
- A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)
- The cost and effort of each strategy compared with its benefit
- Which strategy gives the best value for the hospital

Your recommendation must be supported by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified, cost-aware prevention strategies.

TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.

Discuss:

- The process changes you recommend and their order of priority
- The role of testers, reviews and the defect repository in your plan
- The expected effect on the number and cost of defects
- Trade-offs, risks and limitations of your plan
- Whether the plan remains practical for a small team of five developers

Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.

Evidence Expected

A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Quality Process **8 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.

At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.

Patient health records will now be shared between hospitals.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- What has changed from the original situation
- Which assumptions in your quality improvement plan are affected
- Why frequent releases change the role of testing and the tester
- Why sharing health records raises the severity of some defect classes

Evidence Expected

A clear analysis of how the changed conditions affect the original quality improvement plan.

B. Evaluation of the Original Plan – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended plan remains appropriate.

Discuss:

- Whether the planned reviews and test activities fit into a two-week cycle
- Which new defect origins are introduced by the external health ID system
- Whether your defect repository is still sufficient

Evidence Expected

A technically supported evaluation of the suitability of the original plan under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- Describe the revised process and the tester's revised role
- Explain which testing principles now become more important
- Explain how defects from the external system will be classified and tracked

If you retain the original plan, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate response to the changed requirement with clear quality-engineering reasoning.

D. Evidence-Based Defence – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Use:

- The defect data and cost figures
- Defect origins and classes
- Release frequency
- Patient-safety and data-privacy risk
- Team size and capacity

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

TASK 1 – EVALUATE TESTING AS AN ENGINEERING ACTIVITY

CO1 | K5 – Evaluate | 5 Marks

1.1 Introduction

Software testing is an important engineering activity that ensures a software application meets its requirements, performs correctly and provides reliable services to users. It is not simply the process of running a program and checking whether errors appear. It involves systematic planning, designing test cases, executing tests, measuring results and reporting defects.

In the MediQueue Hospital Appointment Application, patients use the application to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. Approximately 6,000 patients use the application every day. Therefore, the application must provide accuracy, reliability, security and good performance.

However, the development team consisted of only five developers and did not have a separate testing team. Developers tested their own code just before release, and management approved the application based on a successful demonstration.

This approach cannot be considered proper engineering-based testing because quality was not systematically planned, measured or verified throughout development.

1.2 Characteristics of Testing as an Engineering Activity

Testing as an engineering activity consists of several important characteristics.

Characteristic	Description	Importance
Planning	Defining testing objectives, scope, resources and schedule before testing begins.	Ensures testing covers important application features.

Measurement	Collecting information such as defect count, test coverage, failure rate and execution results.	Helps evaluate the actual quality of software.
Standards	Following defined coding, testing, documentation and quality standards.	Maintains consistency and reliability.
Repeatability	Ensuring that the same test can be executed again under similar conditions to obtain consistent results.	Helps identify defects and verify corrections.

A. Planning

Testing planning involves identifying the features to be tested, the risks involved, testing resources and expected results.

Evidence from MediQueue:

- There was no separate tester.
- Developers tested their own code just before release.
- No formal requirement review was conducted.
- The release decision was based on a management demonstration.

Effect on incidents:

Because testing was not planned, important situations were missed. For example, I1 occurred when two patients received the same token number while booking simultaneously. A planned concurrency test could have identified this problem before release.

Similarly, I2 occurred because Sunday appointments were accepted even though the outpatient department was closed. Proper requirement planning and test scenario preparation could have identified this missing business rule.

B. Measurement

Measurement involves collecting and analysing testing information to understand software quality.

Important measurements include:

1. Number of defects identified during each development phase.
2. Number of test cases executed and passed.
3. Number of test cases failed.
4. Defect severity and priority.
5. Application response time.
6. Test coverage percentage.

Evidence from MediQueue:

The project did not demonstrate a systematic measurement-based testing approach. The defect data shows that 60 defects were identified only after production release.

Effect on incidents:

The application took 25 seconds to load the doctor's schedule screen (I10). Performance measurement could have identified this issue before deployment.

Likewise, systematic validation of token generation and payment calculations could have identified I1 and I3.

C. Standards

Standards are predefined rules and guidelines followed during software development and testing.

Examples include:

- Requirement documentation standards.
- Coding standards.
- Test case design standards.

- Defect reporting standards.
- Security and privacy standards.
- Documentation standards.

Evidence from MediQueue:

There was no formal review of requirements or design. The user manual also referred to a menu called "My Visits", which did not exist in the application (I7).

Effect on incidents:

The absence of documentation and review standards resulted in inconsistencies between the actual application and its user manual.

Similarly, the misunderstanding of GST rules caused the application to display ₹500 but charge ₹550 (I3).

D. Repeatability

Repeatability means executing the same test cases under similar conditions and obtaining consistent results.

A test must be documented with input values, execution steps and expected results.

Evidence from MediQueue:

The developers tested their own code just before release. The case does not indicate the existence of a documented regression test suite.

Effect on incidents:

The prescription PDF displayed the previous patient's details when two doctors saved prescriptions simultaneously (I6). Repeated concurrency and data-isolation tests could have detected this defect.

Incident I8 also shows poor repeatability because a refund test case was marked as passed using an incorrect expected value.

1.3 Relationship Between Missing Engineering Characteristics and Incidents

Missing characteristic	**Related incident**	**Impact**
Testing planning	I1, I2	Concurrency and business rules were not properly tested.
Measurement	I10	Performance problems were not measured before release.
Standards	I3, I7	Incorrect payment rules and inconsistent documentation.
Repeatability	I6, I8	Concurrency defects and incorrect test results remained undetected.
Formal review	I2, I3	Missing or misunderstood requirements were not identified.

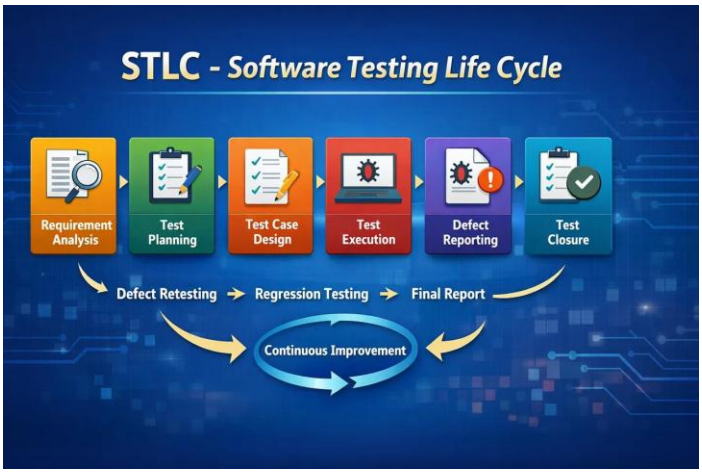
1.4 What an Engineered Testing Approach Would Look Like

MediQueue should adopt a systematic testing approach throughout the software development life cycle.

The following activities should be implemented:

1. Requirement analysis: Review all functional and non-functional requirements, including appointment rules, payment calculations and patient eligibility.
2. Test planning: Prepare a testing strategy, identify risks and allocate responsibilities.
3. Test case design: Create positive, negative, boundary-value and concurrency test cases.
4. Test execution: Perform unit, integration, system, performance and security testing.

5. Defect reporting: Record every defect with its description, severity, priority and reproduction steps.
6. Retesting: Verify that corrected defects no longer occur.
7. Regression testing: Ensure that modifications do not affect existing functionality.
8. Release evaluation: Approve releases only when defined quality criteria are satisfied.



1.5 Evaluation and Conclusion

The MediQueue team did not engineer quality into the application. Instead, it depended mainly on developer self-checking and a successful demonstration. The presence of 60 production defects clearly indicates that the testing approach was insufficient.

Testing should have been planned, measured, standardised and repeated throughout development. An engineering-based testing process would have helped identify important defects before patients experienced them.

Therefore, the MediQueue project cannot be considered an example of testing as a proper engineering activity. The hospital must introduce a structured quality engineering process to improve application reliability and patient satisfaction.

TASK 2 – EVALUATE THE ROLE OF PROCESS IN SOFTWARE QUALITY

CO1 | K5 – Evaluate | 2 Marks

2.1 Introduction

Software quality is influenced by both the technical skills of developers and the processes followed during development. Skilled developers are important, but individual talent alone cannot guarantee a reliable software product.

A defined software process provides a systematic method for analysing requirements, designing the system, writing code, testing functionality and maintaining the application.

In MediQueue, the management believes that hiring better developers will solve the quality problems. However, the actual problem is not only the developers' technical ability but also the absence of a structured development and testing process.

2.2 Role of a Defined Process in Software Quality

A defined process ensures that every development activity follows a planned and controlled approach.

The important roles of a software process are:

1. Requirement management: Ensures that all business rules are documented correctly.
2. Design review: Identifies architectural and logical problems before coding.
3. Coding standards: Maintains consistency and reduces programming errors.
4. Systematic testing: Ensures that all important features are tested.
5. Defect management: Helps identify, record, track and correct defects.
6. Quality monitoring: Measures software quality using defined metrics.
7. Continuous improvement: Uses previous defects to improve future development activities.

For example, if MediQueue had followed a proper requirement review process, the missing Sunday appointment restriction (I2) could have been identified before implementation.

2.3 Would Better Developers Alone Have Prevented the Incidents?

No. Hiring better developers alone would not have prevented all the incidents.

Incident	**Why better developers alone may not prevent it**
I1 – Duplicate token number	Requires concurrency control and systematic testing.
I2 – Sunday booking	The Sunday restriction was never documented in requirements.
I3 – Incorrect consultation fee	GST rules were misunderstood and required clarification.
I5 – Senior-citizen priority	Requires clearly defined age eligibility rules and boundary testing.
I6 – Previous patient's prescription details	Requires proper data isolation and concurrency testing.
I8 – Incorrect refund test result	Requires a verified expected result or test oracle.

Even experienced developers can misunderstand requirements or overlook unusual situations. A defined process helps reduce these risks through reviews, independent testing and verification.

2.4 Process Maturity Using TMM Levels

The Testing Maturity Model (TMM) provides a way to evaluate how systematically an organisation performs software testing.

TMM Level	**Name**	**Description**
Level 1	Initial	Testing is informal and depends mainly on individual effort.

Level 2	Definition	Basic testing policies and planning activities are introduced.
Level 3	Integration	Testing becomes integrated into the software development life cycle.
Level 4	Management and Measurement	Testing activities are measured and controlled using metrics.
Level 5	Optimisation	Continuous improvement and defect prevention become regular activities.



Current Maturity Level of MediQueue

MediQueue is closest to TMM Level 1 – Initial because:

- Testing was informal.
- Developers tested their own code.
- There was no separate tester.
- No formal requirement and design reviews were conducted.
- Testing was performed mainly before release.

- Sixty defects were reported after production deployment.

The team has not established a repeatable and measurable testing process.

2.5 Conclusion

Software quality is not achieved through good developers alone. It requires a combination of skilled people, defined processes, proper testing and continuous improvement.

MediQueue's defects mainly reflect weaknesses in its development and testing process. Therefore, the hospital should focus on improving process maturity rather than depending only on hiring better developers.

TASK 3 – EVALUATE TESTING AS A PROCESS

CO1 | K5 – Evaluate | 6 Marks

3.1 Introduction

Software testing should not be treated as a single activity performed just before releasing an application. It must be a continuous process throughout the software development life cycle.

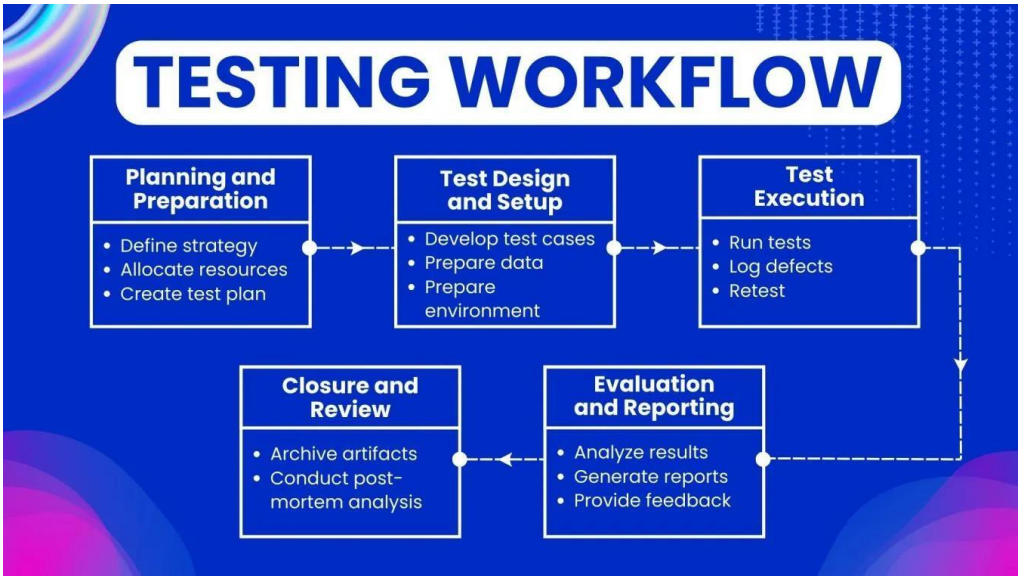
In MediQueue, testing was performed mainly at the end of development. As a result, several defects that originated during requirements, design and coding reached production.

A process-based testing approach involves planning, test design, execution, evaluation and reporting at every suitable development stage.

3.2 Testing as a Continuous Process

The main activities involved in the testing process are explained below.

Testing activity	**Description**	**MediQueue example**
Test planning	Identifying scope, objectives, resources and schedule.	Planning tests for appointments, payments and prescriptions.
Test design	Preparing test cases and expected results.	Creating test cases for Sunday bookings and senior-citizen eligibility.
Test execution	Running test cases using defined inputs.	Testing simultaneous appointment bookings.
Test evaluation	Comparing actual results with expected results.	Checking whether the correct consultation fee is charged.
Test reporting	Recording defects and communicating testing results.	Reporting duplicate tokens and incorrect prescription details.



3.3 Relationship Between Verification and Validation

Verification and validation are two important activities in software quality assurance.

Verification: Checks whether the software is being developed correctly according to specified requirements and design.

Validation: Checks whether the developed software satisfies the actual needs of users.

Development phase	**Verification activity**	**Validation activity**
Requirements	Reviewing requirements for completeness and correctness.	Confirming appointment and payment rules with hospital management.
Design	Inspecting architecture, database design and workflows.	Evaluating whether the design supports hospital operations.
Coding	Code reviews and static analysis.	Unit testing individual functions.
Integration	Reviewing interfaces and data flow.	Testing communication between payment, booking and prescription modules.

System testing	Checking compliance with specifications.	Testing the complete application from a patient's perspective.
Acceptance	Checking release documentation and test results.	Hospital staff and selected users verify that the application meets their needs.

Both verification and validation are necessary to prevent defects from reaching production.

3.4 Phase in Which Each MediQueue Defect Should Have Been Detected

Incident	**Defect description**	**Suitable detection phase**	**Reason**
I1	Duplicate token numbers	Design / Integration testing	Concurrency control and simultaneous booking must be tested.
I2	Sunday appointments accepted	Requirements review	The hospital's Sunday closure rule was missing.
I3	Incorrect consultation fee	Requirements / System testing	GST calculation rules and displayed payment values needed verification.
I4	Application crashes for long names	Coding / Boundary-value testing	Input length limits should be tested.
I5	Senior-citizen priority not applied	Requirements / System testing	Age eligibility on the appointment date must be clearly defined and tested.
I6	Previous patient's prescription details displayed	Design / Integration testing	Data isolation and concurrent database operations must be verified.
I7	User manual refers to a nonexistent menu	Documentation review	Documentation must match the actual application.

I8	Refund test marked Pass with wrong expected value	Test design / Test review	The expected result should be independently verified.
I9	SMS reminders fail after API key change	Integration testing	External service configuration and API failure scenarios must be tested.
I10	Doctor's schedule takes 25 seconds to load	Performance testing	Response time must be measured under realistic conditions.

This table shows that most defects could have been identified before the final testing stage if testing activities had been distributed throughout the life cycle.

3.5 Cost Analysis of Defects

The defect data provided in the assignment shows that the cost of fixing defects increases significantly when they are detected late.

A. Cost of Fixing Defects in Each Phase

The following calculations use the defect counts and cost per defect given in the assignment.

Formula:

Total Fixing Cost = Number of Defects × Cost per Defect

Phase	**Defects detected**	**Cost per defect**	**Total fixing cost**
Requirements	2	₹500	₹1,000
Design	3	₹1,500	₹4,500
Coding	15	₹3,000	₹45,000
Testing	20	₹8,000	₹1,60,000
Production	60	₹40,000	₹24,00,000

Total	100	–	₹26,10,500

Cost of fixing defects by detection phase

Calculated using the assignment's defect counts and cost per defect.

₹0₹650K₹1.3M₹2M₹2.6MRequirementsDesignCodingTestingProduction

Interpretation:

- Requirements defects cost only ₹500 per defect.
- Design defects cost ₹1,500 per defect.
- Coding defects cost ₹3,000 per defect.
- Testing defects cost ₹8,000 per defect.
- Production defects cost ₹40,000 per defect.

The total estimated fixing cost is ₹26,10,500. Out of this, production defects alone account for ₹24,00,000.

Therefore, late defect detection has a major financial impact on the hospital.

B. Cost of Production Defects Compared with Early Detection

If all 60 production defects had been detected during the requirements phase, the estimated cost would have been:

60 × ₹500 = ₹30,000

Actual production fixing cost:

60 × ₹40,000 = ₹24,00,000

Potential difference:

₹24,00,000 − ₹30,000 = ₹23,70,000

Estimated difference: ₹23,70,000

This illustrates the financial advantage of detecting defects early.

Important assumption: This is a theoretical comparison. It assumes that all 60 production defects could have been identified during the requirements phase and fixed at the requirements-phase cost. In reality, some defects originate during design, coding or integration and cannot all be detected through requirements reviews.

3.6 Likely Savings Through Requirements and Design Reviews

The assignment identifies 30 defects introduced during requirements and 22 defects introduced during design.

If formal reviews had been conducted, some of these defects could have been prevented or detected before coding.

For an illustrative estimate, assume:

- Requirements reviews prevent or detect 70% of the 30 requirements defects.
- Design reviews prevent or detect 60% of the 22 design defects.

Requirements review:

30 × 70% = 21 defects

Estimated saving:

21 × ₹500 = ₹10,500

Design review:

22 × 60% = 13.2 defects

Estimated saving:

13.2 × ₹1,500 = ₹19,800

Total estimated saving:

₹10,500 + ₹19,800 = ₹30,300

Thus, the hospital could potentially save approximately ₹30,300 in direct defect-fixing costs under these assumptions.

This estimate does not include the additional savings from avoiding rework in later phases, production disruption, patient complaints or reputational damage.

3.7 Limitations of the Cost Estimate

The calculation has certain limitations:

1. The review effectiveness percentages are assumptions, not measured values from the case study.
2. It assumes that all prevented or detected defects have the same fixing cost within their respective phases.
3. It does not include the cost of conducting formal reviews.
4. Some defects may have multiple origins.
5. Production defects may create additional indirect costs, including patient dissatisfaction and operational disruption.
6. Not every defect can be identified during requirements or design reviews.

Therefore, the calculated savings should be treated as an illustrative estimate rather than a guaranteed financial result.

3.8 Conclusion

Testing must be integrated into every phase of software development rather than being postponed until the end. MediQueue's defect data shows that 60 defects were detected only after production release, resulting in an estimated fixing cost of ₹24,00,000.

Requirements reviews, design inspections, coding checks, integration tests and system testing could have identified many defects earlier.

Therefore, the hospital should adopt a continuous testing process to reduce defect leakage, improve software reliability and minimise maintenance costs.

TASK 4 – APPLY AND CRITIQUE BASIC DEFINITIONS

CO1 | K5 – Evaluate | 4 Marks

4.1 Introduction

Software testing uses several basic terms to identify problems, understand their causes and evaluate application behaviour. Terms such as error, fault, failure, test case, test oracle and test bed have different meanings.

In MediQueue, the development team uses the words "bug", "error" and "failure" interchangeably. This creates confusion when reporting defects and identifying their actual causes.

A common understanding of testing terminology is necessary to improve communication between developers, testers and hospital management.

4.2 Basic Testing Definitions in the MediQueue Context

Term	Meaning	Example in MediQueue
Error	A human mistake made during requirements analysis, design, coding or testing.	A developer misunderstands the GST calculation rule.
Fault (Defect)	A mistake in the software or its documentation that can cause incorrect behaviour.	The payment module calculates or displays the consultation fee incorrectly.
Failure	An observable incorrect behaviour when the software is executed.	The booking page displays ₹500, but the patient is charged ₹550.
Test Case	A set of inputs, conditions, execution steps and expected results used to verify a feature.	Entering a patient's details and checking whether the correct appointment token is generated.

Test Oracle	A source or mechanism used to determine the correct expected result of a test.	The hospital-approved GST rule used to verify the final consultation fee.
Test Bed	The environment, hardware, software, tools and test data used to execute testing.	A test server with a database, payment gateway sandbox and sample patient records.

4.3 Application of Error, Fault and Failure to Incidents I3, I5 and I6

A. Incident I3 – Incorrect Consultation Fee

Incident: The booking page displays ₹500, but the application charges ₹550 because the GST rule was misunderstood.

- Error: The developer or requirements analyst misunderstood the hospital's GST calculation rule.
- Fault: The payment module contains an incorrect fee calculation or inconsistent fee-display logic.
- Failure: The patient sees ₹500 on the booking page but is charged ₹550.

Evaluation:

This incident demonstrates how a human misunderstanding can become a software defect and eventually affect patients financially. Proper requirement clarification and payment validation could have prevented this problem.

B. Incident I5 – Senior-Citizen Priority

Incident: Senior-citizen priority is not applied to patients who turn 60 on the appointment date.

- Error: The eligibility rule was incorrectly understood or interpreted.
- Fault: The application uses an incorrect age calculation or eligibility condition.

- Failure: A patient who becomes eligible on the appointment date does not receive senior-citizen priority.

Evaluation:

This incident shows the importance of boundary-value testing. The system should verify age eligibility using the appointment date rather than relying only on the patient's age on the booking date.

C. Incident I6 – Previous Patient's Prescription Details

Incident: The prescription PDF displays the previous patient's details when two doctors save prescriptions simultaneously.

- Error: The developer failed to properly consider simultaneous prescription operations and patient-data isolation.
- Fault: The application contains an incorrect shared-data handling or concurrency-control mechanism.
- Failure: A doctor receives a prescription PDF containing another patient's information.

Evaluation:

This is a serious defect because it affects patient privacy and the accuracy of medical records. Proper database transaction management, data isolation and concurrent-user testing are essential to prevent this problem.

4.4 Why Terminology Confusion Made Incident I8 Possible

Incident I8: A refund test case was marked as "Pass" because the tester compared the output with an incorrect expected value.

A test case must contain a correct expected result. The expected result should be derived from an approved requirement or another reliable test oracle.

In MediQueue, the tester used an incorrect expected value and still marked the test as passed.

The confusion between an error in the test itself and a defect in the application caused the testing result to become unreliable.

For example:

Testing element	**Example**
Test objective	Verify the refund amount.
Input	A payment of ₹550 is refunded.
Correct expected result	₹550, assuming a full refund is permitted by the refund policy.
Incorrect expected result	₹500.
Actual result	₹500.
Incorrect test decision	Pass.
Correct test decision	Fail, because the expected value was wrong.

The test case cannot be trusted simply because the actual output matches the expected output. The expected output must also be correct.

4.5 Need for a Common Testing Glossary

MediQueue should maintain a common testing glossary so that all team members use the same terminology.

The glossary should contain:

1. Error – A human mistake during development or testing.
2. Fault – A defect in software or documentation.
3. Failure – Incorrect behaviour observed during execution.
4. Test case – Defined inputs, conditions and expected results.
5. Test oracle – A reliable source for determining expected results.
6. Test bed – The environment used for testing.

7. Severity – The impact of a defect on the application or users.
8. Priority – The urgency with which a defect must be fixed.
9. Retesting – Testing a corrected defect again.
10. Regression testing – Checking whether changes have affected existing functionality.

4.6 Conclusion

Testing terminology is not merely a matter of words. It directly affects defect reporting, communication, test execution and quality decisions.

The incidents I3, I5 and I6 demonstrate how human errors can become software faults and lead to failures. Incident I8 shows that an incorrect test oracle can produce misleading test results.

Therefore, MediQueue should establish a common testing glossary and ensure that developers, testers and management follow consistent terminology.

TASK 5 – EVALUATE THE APPLICATION OF TESTING PRINCIPLES

CO1 | K5 – Evaluate | 3 Marks

5.1 Introduction

Software testing principles are basic guidelines that help testers perform testing effectively and evaluate software quality. These principles explain the limitations of testing and help identify important testing activities.

The MediQueue developers claim that "we tested everything and found no errors, so the software was correct."

This claim is incorrect because testing cannot prove that software is completely free from defects. The ten incidents reported after release clearly demonstrate that the testing approach was insufficient.

5.2 Important Software Testing Principles Applied to MediQueue

1. Testing Shows the Presence of Defects, Not Their Absence

Testing can identify defects, but it cannot prove that an application contains no defects.

MediQueue example:

The developers believed that the application was correct because their testing and management demonstration showed no errors. However, 60 defects were reported after production release.

Evaluation:

A successful demonstration does not guarantee that every feature works correctly under all conditions. Therefore, the developers' claim is invalid.

2. Exhaustive Testing Is Impossible

It is not practical to test every possible input, condition and combination in a complex application.

MediQueue example:

Incident I1 occurred when two patients booked appointments at the same second and received the same token number.

Incident I4 occurred when a patient name exceeded 40 characters.

Evaluation:

The team should have selected important test scenarios, including boundary values, simultaneous bookings and invalid inputs, based on risk and business requirements.

3. Early Testing Saves Time and Money

Testing should begin during requirements analysis and continue throughout development.

MediQueue example:

Incident I2 occurred because the Sunday closure rule was never written in the requirements. Incident I3 occurred because the GST rule was misunderstood.

Evaluation:

Requirement reviews and design inspections could have identified these problems before implementation. Early detection would have reduced correction costs and prevented defects from reaching patients.

4. Defect Clustering

A small number of modules often contain a large proportion of software defects. Therefore, testing effort should focus on modules with higher defect concentration and risk.

MediQueue example:

The application contains important modules such as:

- Appointment booking.
- Token generation.
- Payment processing.
- Prescription management.
- SMS notifications.

Incidents I1, I3 and I6 indicate problems in important booking, payment and prescription functionality.

Evaluation:

The hospital should prioritise testing of high-risk modules instead of distributing testing effort equally across all application features.

5. Pesticide Paradox

Repeatedly executing the same test cases may eventually stop identifying new defects. Test cases must be reviewed and updated regularly.

MediQueue example:

The developers performed self-testing before release, but important situations such as simultaneous bookings and long patient names were not identified.

Evaluation:

The team should regularly introduce new test scenarios, boundary-value tests, negative tests and concurrency tests to identify defects that existing tests may miss.

6. Testing Is Context Dependent

Testing methods and priorities depend on the type of application, its users and the risks associated with failures.

MediQueue example:

MediQueue is a healthcare appointment application used by approximately 6,000 patients every day.

A defect in prescription handling (I6) is more serious than a minor documentation issue because it can expose confidential patient information.

Evaluation:

The hospital should give greater importance to data accuracy, privacy, payment correctness, appointment reliability and performance.

5.3 Summary of Testing Principles and Case Evidence

Testing principle	**Related incident**	**Evaluation**
Testing shows the presence of defects, not their absence	I1–I10	Successful demonstrations did not prove the application was defect-free.
Exhaustive testing is impossible	I1, I4	Important combinations and boundary conditions were missed.
Early testing saves time and money	I2, I3	Requirement and design problems were detected too late.
Defect clustering	I1, I3, I6	High-risk application modules need more testing attention.
Pesticide paradox	I1, I4	Repeating limited self-tests was insufficient to identify new defects.
Testing is context dependent	I6, I9, I10	Healthcare, integration and performance risks require specialised testing.

5.4 Most Important Principle for MediQueue

The principle that would have the largest overall effect on MediQueue's quality is Early Testing Saves Time and Money.

The main reasons are:

1. The project had no formal requirement reviews.
2. Thirty defects originated during requirements.
3. Twenty-two defects originated during design.
4. Sixty defects were detected only after production release.
5. Production defects cost ₹40,000 per defect compared with ₹500 during the requirements phase.

If the team had conducted requirement reviews, design inspections and early test planning, many defects could have been prevented or identified before deployment.

This would have reduced correction costs and improved the reliability of the application.

5.5 Conclusion

The claim that MediQueue was correct because developers tested everything is not valid. Software testing cannot guarantee the complete absence of defects, and exhaustive testing is practically impossible.

The team should follow important testing principles such as early testing, defect clustering, the pesticide paradox and context-dependent testing.

Among these, early testing is the most important for MediQueue because it can reduce defect leakage, minimise correction costs and protect patients from application failures.

TASK 6 – EVALUATE THE TESTER'S ROLE

CO1 | K5 – Evaluate | 3 Marks

6.1 Introduction

A software tester plays an important role in ensuring that an application meets its functional and non-functional requirements. A tester is responsible for identifying defects, verifying software behaviour, evaluating test results and ensuring that the application is reliable before release.

In the MediQueue project, there was no separate independent tester. The five developers tested their own code just before release, and the application was approved based on a successful demonstration to hospital management.

This approach was insufficient because developers may overlook defects in their own code due to familiarity with the implementation. An independent tester could have identified many problems before the application reached patients.

6.2 Role and Responsibilities of a Tester Across the Software Development Life Cycle

A tester should participate in every stage of software development rather than joining only during the final testing phase.

Development phase	**Tester responsibilities**	**MediQueue application**
Requirement analysis	Review requirements and identify missing or unclear conditions.	Verify Sunday closure rules and senior-citizen eligibility.
Design phase	Review system architecture, database design and application workflows.	Check token generation and prescription data isolation.
Coding phase	Prepare unit test scenarios and review code quality.	Validate patient name length and payment calculations.

Integration phase	Verify communication between different modules and external services.	Test payment gateway and SMS API integration.
System testing	Test the complete application against requirements.	Test appointment booking, payments and prescriptions.
Acceptance testing	Verify whether the application satisfies actual user needs.	Allow hospital staff to validate appointment workflows.
Maintenance	Retest corrections and perform regression testing.	Verify that corrected defects do not affect existing features.

6.3 Advantages and Limitations of Developers Testing Their Own Code

Developers testing their own code can be useful, but it should not replace independent testing.

Advantages	**Limitations**
Developers understand the internal code structure.	They may overlook mistakes caused by their own assumptions.
Defects can be corrected immediately.	They may test only expected situations.
Unit testing can be performed quickly.	They may not consider the complete user perspective.
Development and testing can happen together.	They may unintentionally use incorrect expected results.
It reduces communication delays.	Independent verification may be missing.

Example from MediQueue:

In incident I8, the refund test case was marked as passed because the tester compared the output with an incorrect expected value.

An independent tester reviewing the test case and verifying the expected refund amount against the hospital's approved refund policy could have identified this problem.

Therefore, developers should perform unit testing, while independent testers should perform additional verification and validation.

6.4 Incidents an Independent Tester Could Have Detected

Incident	Possible testing activity	Stage of detection
I1 – Duplicate token numbers	Simultaneous booking and concurrency testing	Integration testing
I2 – Sunday appointments accepted	Requirement review and negative testing	Requirement analysis
I3 – Incorrect consultation fee	Payment calculation and GST validation	System testing
I4 – Application crashes for long names	Boundary-value and input validation testing	Unit testing
I5 – Senior-citizen priority missing	Age boundary testing	System testing
I6 – Previous patient's prescription details	Concurrent-user and data-isolation testing	Integration testing
I7 – Incorrect user manual	Documentation review	Documentation testing
I8 – Incorrect refund expected value	Test case review and oracle validation	Test design
I9 – SMS reminders fail after API key change	API integration and configuration testing	Integration testing
I10 – Schedule takes 25 seconds to load	Load and performance testing	Performance testing

6.5 Qualities Required for a Healthcare Application Tester

A tester working on a healthcare application such as MediQueue must possess the following qualities:

1. Technical knowledge: Must understand software testing techniques, databases, APIs and application architecture.
2. Attention to detail: Must carefully examine patient information, appointment dates, token numbers and payment calculations.
3. Analytical thinking: Must identify unusual situations and determine the root cause of failures.
4. Security awareness: Must understand patient privacy, access control and secure handling of medical records.
5. Communication skills: Must communicate defects clearly to developers, hospital staff and management.
6. Domain knowledge: Must understand appointment scheduling, consultation charges, patient eligibility and prescription workflows.
7. Independent thinking: Must test the application from the user's perspective instead of relying only on developer assumptions.

6.6 Evaluation: Would an Independent Tester Have Changed the Outcome?

Yes. An independent tester would have significantly improved the quality of MediQueue.

The tester could have identified missing requirements, incorrect calculations, concurrency problems, input validation issues and performance defects before release.

However, an independent tester alone cannot guarantee that all defects will be prevented. The tester requires proper requirements, sufficient time, a suitable test environment and cooperation from developers.

For a small team of five developers, the hospital does not necessarily need a large separate testing department. It can appoint one dedicated quality engineer who coordinates testing while developers continue performing unit tests.

6.7 Conclusion

The absence of an independent tester was one of the important weaknesses in the MediQueue development process. Developer self-testing was insufficient to identify all the defects affecting patients.

An independent tester should participate throughout the software development life cycle and focus on requirement validation, test design, integration testing, security and performance testing.

Therefore, appointing a dedicated quality engineer and introducing independent testing activities would improve MediQueue's reliability, safety and overall software quality.

TASK 7 – TRACE THE ORIGINS OF DEFECTS

CO1 | K5 – Evaluate | 4 Marks

7.1 Introduction

Defect origin analysis is the process of identifying the stage of software development where a defect was introduced.

A defect may originate during requirements analysis, system design, coding, testing, documentation or environmental configuration.

Identifying the origin of a defect is important because fixing the visible problem alone may not prevent similar defects from occurring again.

In MediQueue, ten major incidents were reported after release. These incidents indicate weaknesses in multiple development stages.

7.2 Classification of Defect Origins

The following categories can be used to classify the incidents:

Defect origin	**Description**
Requirements defects	Missing, incorrect or misunderstood business requirements.
Design defects	Problems in architecture, database structure or system workflows.
Coding defects	Programming mistakes and incorrect implementation.
Testing defects	Incorrect test cases, expected results or test execution.
Documentation defects	Incorrect or outdated user manuals and technical documents.
Environment defects	Configuration, external services, network or deployment-related problems.

7.3 Analysis of All Ten MediQueue Incidents

Incident	Defect origin	Evidence and justification
I1 – Duplicate token number	Design / Coding	Token generation did not properly handle simultaneous bookings.
I2 – Sunday appointments accepted	Requirements	The Sunday closure rule was never included in the requirements.
I3 – Incorrect consultation fee	Requirements / Coding	GST rules were misunderstood and incorrectly implemented.
I4 – Crash for long patient names	Coding	Input length validation and handling were insufficient.
I5 – Senior-citizen priority missing	Requirements / Coding	The age eligibility condition was not correctly interpreted or implemented.
I6 – Previous patient's prescription details	Design / Coding	Concurrent operations did not maintain proper patient-data isolation.
I7 – Incorrect user manual	Documentation	The manual referred to a menu that did not exist.
I8 – Incorrect refund test result	Testing	The expected result was incorrect, but the test was marked as passed.
I9 – SMS reminders fail after API key change	Environment / Integration	The application did not properly handle the SMS provider's changed API configuration.
I10 – Schedule takes 25 seconds to load	Design / Coding	Database queries, application processing or resource usage may be inefficient.

Note: Some incidents have more than one possible origin. The classification identifies the most likely source based on the evidence supplied in the assignment.

7.4 Detailed Analysis of Important Defect Origins

A. Requirements-Origin Defects

Requirements defects occur when business rules are missing, unclear or incorrectly understood.

Related incidents: I2, I3 and I5.

- I2: The Sunday closure rule was not documented.
- I3: The GST calculation rule was misunderstood.
- I5: The age eligibility rule was not correctly interpreted.

These defects could have been reduced through requirement reviews, discussions with hospital management and clearly documented business rules.

B. Design-Origin Defects

Design defects occur when the architecture or system structure cannot correctly support the required functionality.

Related incidents: I1 and I6.

- I1: The token-generation design did not safely handle simultaneous bookings.
- I6: The prescription management design did not ensure proper isolation between simultaneous operations.

These problems require design reviews, transaction management and concurrency testing.

C. Coding-Origin Defects

Coding defects occur when developers implement functionality incorrectly.

Related incidents: I4 and potentially I3, I5 and I10.

- I4: The application crashed when patient names exceeded 40 characters.
- I3: Incorrect payment calculation logic may have been implemented.
- I5: The eligibility condition may have been incorrectly coded.
- I10: Inefficient queries or processing logic may have caused slow performance.

Code reviews, input validation and unit testing can reduce these defects.

D. Testing-Origin Defects

Testing defects occur when the test process itself is incorrect or insufficient.

Related incident: I8.

The tester used an incorrect expected refund value. This made the test result unreliable.

The organisation should independently review test cases and verify expected results using approved business rules.

E. Documentation-Origin Defects

Related incident: I7.

The user manual instructed patients to cancel appointments through a "My Visits" menu that did not exist.

This indicates a lack of documentation review and version control.

F. Environment and Integration Defects

Related incident: I9.

The SMS reminders stopped working when the provider changed its API key.

This indicates that external service configuration and credential changes were not handled properly.

The team should use configuration management, secure secret storage and integration tests.

7.5 Overall Pattern of Defect Origins

The defect data supplied in the assignment shows the following distribution of introduced defects:

Origin phase	Defects introduced	Percentage
Requirements	30	30%
Design	22	22%
Coding	38	38%
Testing	5	5%
Documentation	5	5%
Total	100	100%

Distribution of introduced defects

Based on the defect-origin data in the MediQueue assignment.

Coding
Design
Documentation
Requirements
Testing

Interpretation:

- Coding contributes the largest number of introduced defects (38%).
- Requirements contribute 30% of defects.
- Design contributes 22% of defects.
- Requirements and design together account for 52% of all introduced defects.
- Testing and documentation account for the remaining 10%.

This indicates that MediQueue must improve its requirement analysis, design reviews and coding practices.

7.6 Where Should the Hospital Invest More Effort?

The hospital should prioritise the following improvements:

1. Requirement reviews to identify missing business rules.

2. Design inspections to identify concurrency and database problems.
3. Coding standards and peer code reviews.
4. Independent testing of critical application modules.
5. Integration testing for external services.
6. Documentation verification before every release.

Although coding has the highest number of introduced defects, improving only coding practices will not solve the entire quality problem. More than half of the introduced defects originated during requirements and design.

7.7 Conclusion

The MediQueue defect analysis shows that software problems can originate at different stages of development. Requirements, design and coding are the major sources of defects.

The hospital should not focus only on correcting production failures. It must identify and eliminate the root causes through requirement reviews, design inspections, coding standards and systematic testing.

A defect prevention approach will help reduce repeated problems and improve the reliability of future MediQueue releases.

TASK 8 – EVALUATE DEFECT CLASSES AND SEVERITY

CO1 | K5 – Evaluate | 4 Marks

8.1 Introduction

Defect classification is the process of grouping software defects according to their nature, origin and impact.

A defect classification scheme helps an organisation understand which types of defects occur frequently and which ones create the greatest risk.

In MediQueue, defects affect appointment booking, payment processing, prescription management, SMS notifications and application performance.

Since the application is used in a healthcare environment, defects involving patient safety, privacy and financial transactions must receive special attention.

8.2 Suitable Defect Classification

The following classification can be used for MediQueue.

Defect class	**Sub-class**	**Description**
Requirements defects	Missing requirements	Important business rules are not documented.
Requirements defects	Incorrect requirements	Requirements contain wrong information.
Design defects	Architecture defects	System structure cannot handle required operations correctly.
Design defects	Database defects	Data storage, isolation or transaction handling is incorrect.
Coding defects	Logic defects	Program produces incorrect results.

Coding defects	Input validation defects	Application fails to handle invalid or unexpected inputs.
Testing defects	Test design defects	Test cases contain incorrect expected results or insufficient conditions.
Testing defects	Coverage defects	Important scenarios are not tested.
Documentation defects	User manual defects	Instructions do not match the actual application.
Environment defects	Configuration defects	Incorrect settings or external service changes affect application behaviour.

8.3 Classification, Severity and Priority of All Ten Incidents

Severity indicates how seriously a defect affects the application or users.

Priority indicates how urgently the defect should be corrected.

For this analysis, the following levels are used:

- Critical: Major patient-safety, privacy or serious operational risk.
- High: Significant impact on important application functionality.
- Medium: Limited functionality or usability impact.
- Low: Minor inconvenience or documentation issue.

Incident	**Defect class**	**Severity**	**Priority**
I1 – Duplicate token number	Design / Coding	High	High
I2 – Sunday appointments accepted	Requirements	High	High
I3 – Incorrect consultation fee	Requirements / Coding	High	High
I4 – Crash for long patient names	Coding	High	High
I5 – Senior-citizen priority missing	Requirements / Coding	High	High
I6 – Previous patient's prescription details	Design / Coding	Critical	Critical
I7 – Incorrect user manual	Documentation	Low	Medium

I8 – Incorrect refund expected value	Testing	High	High
I9 – SMS reminders fail after API key change	Environment / Integration	Medium	High
I10 – Schedule takes 25 seconds to load	Design / Coding	High	High

The severity and priority levels above are proposed engineering judgements for this assignment, not classifications supplied by the hospital.

8.4 Detailed Justification of Important Defects

A. I1 – Duplicate Token Numbers

This is a high-severity defect because two patients may receive the same token for the same doctor.

It can create confusion, delays and appointment conflicts in the hospital.

The defect should be assigned high priority because token generation is a core application feature.

B. I3 – Incorrect Consultation Fee

This defect affects the financial accuracy of the application.

Patients see ₹500 but are charged ₹550. This can result in complaints, refund requests and loss of trust.

Therefore, it should be treated as a high-severity and high-priority defect.

C. I6 – Previous Patient's Prescription Details

This is a critical defect because one patient's confidential medical information may be displayed in another patient's prescription.

It affects data privacy, medical-record accuracy and patient trust.

The hospital should immediately investigate and correct this defect before continuing to use the affected prescription workflow.

D. I7 – Incorrect User Manual

This is a documentation defect because the manual refers to a menu that does not exist.

It creates confusion for patients but does not directly affect core application processing.

Therefore, it may have low severity but medium priority because correcting the instructions is relatively simple and improves usability.

E. I9 – SMS Reminders Fail

This is an integration and configuration defect.

Patients may not receive appointment reminders when the SMS provider changes its API key.

Although the application may continue to support appointment booking, missing reminders can cause inconvenience and missed appointments.

Therefore, it has medium severity but high priority because the issue can be corrected quickly and affects patient communication.

8.5 Difference Between Severity and Priority

Severity and priority are related but they are not the same.

Severity	**Priority**
Measures the impact of a defect.	Measures how urgently the defect must be fixed.
Based on technical and business consequences.	Based on urgency, risk and operational needs.

Usually assessed during defect analysis.	May change according to release plans and business decisions.
Example: I6 is critical because it exposes patient data.	I6 must receive immediate correction priority.

Example – I7 and I1:

I7 is a documentation problem that confuses patients. It has low severity, although it should still be corrected.

I1 affects appointment token generation and can disrupt hospital operations. It has high severity and high priority.

A low-severity defect can sometimes receive high priority if it affects a large number of users or blocks an important release. Similarly, a high-severity defect must be assessed immediately even if the affected feature is used infrequently.

8.6 Which Defect Class Should the Team Target First?

The hospital should first target defects related to patient data accuracy, privacy and critical system functionality.

The recommended order is:

1. Design and database defects affecting patient records and data isolation.
2. Requirements defects affecting appointment rules and financial calculations.
3. Coding defects affecting application stability and correctness.
4. Integration and environment defects affecting external services.
5. Testing defects affecting the reliability of test results.
6. Documentation defects affecting user understanding.

This order prioritises the potential harm to patients rather than simply the number of defects.

8.7 Conclusion

Defect classification helps MediQueue identify the nature and seriousness of software problems. The incident analysis shows that patient-data privacy and prescription accuracy are the most critical concerns.

The hospital should use severity and priority to organise defect correction activities. Critical defects must be addressed immediately, while other defects should be corrected according to their impact and urgency.

Therefore, a structured defect classification system will help the hospital allocate testing and development resources more effectively.

TASK 9 – DESIGN AND APPRAISE A DEFECT REPOSITORY

CO1 | K5 – Evaluate | 3 Marks

9.1 Introduction

A defect repository is a centralised system used to record, track, analyse and manage software defects throughout the software development life cycle.

It stores information about reported defects, their causes, severity, priority, correction status and testing results.

Currently, MediQueue records patient complaints through emails and spreadsheets. This approach makes it difficult to track defects, identify repeated problems and understand the overall quality of the application.

The hospital should introduce a structured defect repository to improve defect management and prevent similar problems in future releases.

9.2 Important Fields in a Defect Repository

A defect repository must contain sufficient information to help developers and testers understand, reproduce and correct each defect.

Field	Description	Example in MediQueue
Defect ID	Unique identification number for each defect.	MQ-001
Defect Title	Short description of the problem.	Duplicate token generation
Description	Detailed explanation of the defect.	Two patients receive the same token.
Module	Application component where the defect occurred.	Appointment Booking
Defect Origin	Development phase where the defect was introduced.	Design

Defect Class	Type of defect.	Concurrency defect
Severity	Impact of the defect.	High
Priority	Urgency of correction.	High
Environment	Device, browser, operating system or server details.	Android application, hospital Wi-Fi
Steps to Reproduce	Instructions to recreate the defect.	Book appointments simultaneously for the same doctor.
Expected Result	Correct behaviour of the application.	Each patient receives a unique token.
Actual Result	Behaviour observed during testing.	Both patients receive the same token.
Reported By	Person who identified the defect.	Quality Engineer
Assigned To	Developer responsible for correction.	Backend Developer
Status	Current state of the defect.	Open / In Progress / Fixed / Closed
Root Cause	Actual reason for the defect.	Missing concurrency control
Fix Details	Description of the correction.	Implemented atomic token generation.
Retest Result	Result after correction.	Passed
Date Reported	Date when the defect was identified.	04-11-2026
Date Closed	Date when the defect was resolved.	06-11-2026

9.3 Sample Defect Repository Records

The following table demonstrates how MediQueue can maintain defect information.

Defect ID	Incident	Module	Severity	Status	Corrective action
MQ-001	I1 – Duplicate token	Appointment	High	Open	Implement concurrency control.

MQ-002	I3 – Incorrect fee	Payment	High	Open	Correct GST calculation and validation.
MQ-003	I6 – Wrong prescription details	Prescription	Critical	Open	Fix patient-data isolation.
MQ-004	I7 – Incorrect manual	Documentation	Low	Open	Update user instructions.
MQ-005	I9 – SMS failure	Notification	Medium	Open	Improve API key configuration handling.

Illustrative repository entries; statuses are examples rather than actual hospital records.

9.4 How a Defect Repository Supports Software Quality

A. Defect Tracking

Every defect receives a unique identification number and a status.

The team can determine whether a defect is open, assigned, under correction, fixed or closed.

For example, the hospital can track whether the duplicate token problem has been corrected and independently verified.

B. Root-Cause Analysis

A repository stores the origin and root cause of every defect.

By analysing repeated problems, the hospital can identify whether defects mainly originate from requirements, design, coding or testing.

C. Defect Trend Analysis

The repository allows the team to prepare reports showing:

- Total defects reported.

- Defects by application module.
- Defects by severity.
- Defects by development phase.
- Defects fixed during each release.
- Defects reopened after correction.

These reports help management understand whether software quality is improving.

D. Process Improvement

If the repository repeatedly identifies missing requirements, the hospital can improve its requirement review process.

If multiple defects occur in payment calculations, additional financial validation and independent testing can be introduced.

E. Knowledge Sharing

Developers and testers can refer to previously reported defects and their solutions.

This reduces repeated mistakes and helps new team members understand common application problems.

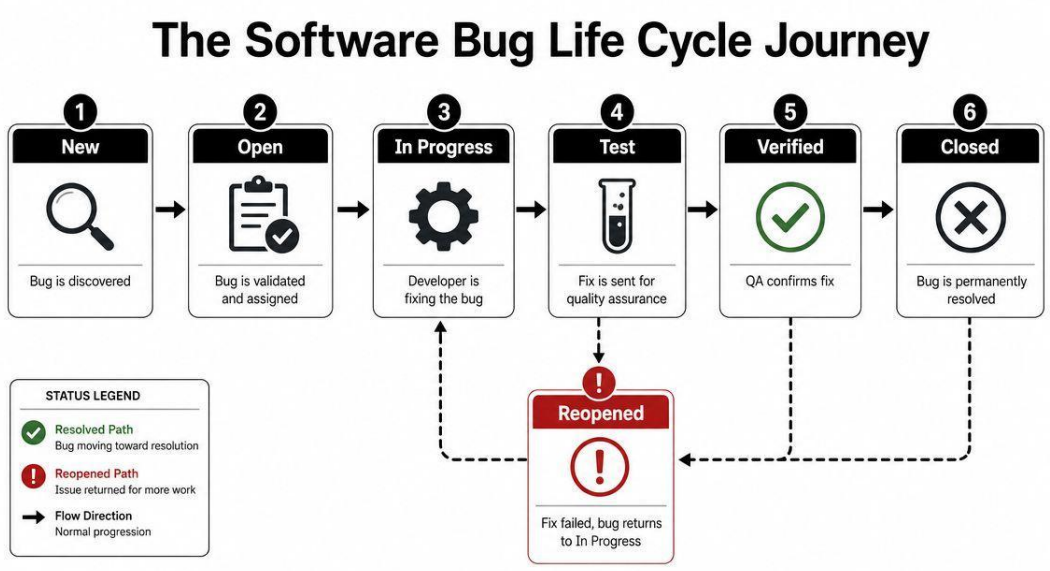
9.5 Weaknesses of the Existing Email and Spreadsheet Approach

Existing method	**Limitation**	**Recommended improvement**
Email complaints	Important defects may get lost among messages.	Use a centralised defect repository.
Manual spreadsheets	Multiple copies may contain different information.	Maintain one controlled database.
No unique defect ID	Difficult to refer to individual defects.	Generate unique defect numbers.

No proper status tracking	Management cannot easily identify unresolved issues.	Introduce defect lifecycle statuses.
Limited analysis	Difficult to identify recurring defect patterns.	Generate defect reports and dashboards.
No structured root-cause details	The same mistakes may happen again.	Record root causes and preventive actions.

9.6 Recommended Defect Management Workflow

The following workflow can be implemented in MediQueue:



Workflow explanation:

1. Defect identification: A tester, developer or hospital user identifies a problem.
2. Defect logging: The defect is recorded with a unique ID and supporting information.
3. Defect classification: The team identifies its origin, class, severity and priority.
4. Defect assignment: The issue is assigned to the responsible developer.
5. Defect correction: The developer investigates and fixes the problem.
6. Retesting: The tester verifies the correction.

7. Regression testing: Related application features are tested again.
8. Defect closure: The defect is closed after successful verification.

If the defect still exists, it should be reopened and returned for further correction.

9.7 Recommended Repository Tools

MediQueue can use a defect-tracking tool such as:

- Jira – For defect tracking, assignment and workflow management.
- GitHub Issues – For connecting defects with code changes and development tasks.
- Bugzilla – For recording and tracking software defects.

For a small team of five developers, GitHub Issues or a simple Jira project can provide sufficient defect-tracking functionality.

9.8 Conclusion

A defect repository is essential for systematic software quality management. The existing email and spreadsheet approach does not provide sufficient control over defect identification, tracking and analysis.

By implementing a centralised repository, MediQueue can identify repeated defects, monitor correction activities, analyse defect trends and improve future development processes.

Therefore, the hospital should replace informal complaint tracking with a structured defect management system.

TASK 10 – EVALUATE DEFECT EXAMPLES AND PREVENTION STRATEGIES

CO1 | K5 – Evaluate | 4 Marks

10.1 Introduction

Defect prevention is the process of identifying and eliminating the root causes of software defects before they affect users.

Correcting a defect after release may solve the immediate problem, but it does not always prevent similar defects from occurring again.

MediQueue experienced several defects because of weaknesses in requirements, design and testing. Therefore, the hospital should introduce preventive activities at the appropriate development stages.

For this analysis, three incidents from different defect classes are selected:

- I2 – Requirements defect.
- I6 – Design and data-isolation defect.
- I8 – Testing defect.

10.2 Selected Defects and Their Root Causes

A. Incident I2 – Sunday Appointments Accepted

Requirements defect

Problem:

The application accepts appointments for Sunday even though the hospital's outpatient department remains closed.

Root cause:

The Sunday closure rule was not included in the software requirements.

The development team implemented appointment booking without receiving complete business rules from hospital management.

Prevention strategy: Formal Requirement Review

The hospital should introduce a formal requirement review process before development begins.

The review should include:

1. Discussions with hospital management.
2. Collection of appointment scheduling rules.
3. Identification of working days and holidays.
4. Preparation of positive and negative test scenarios.
5. Approval of requirements by authorised stakeholders.

Example test case:

Test input	**Expected result**
Appointment date: Monday	Booking accepted.
Appointment date: Saturday	Booking accepted if the department operates on Saturday.
Appointment date: Sunday	Booking rejected with a suitable message.

Cost and benefit:

The cost of requirement reviews is relatively low because they mainly require meetings, document inspections and stakeholder participation.

The benefit is high because missing business rules can be identified before coding begins.

B. Incident I6 – Previous Patient's Prescription Details

Critical design defect

Problem:

The prescription PDF displays the previous patient's details when two doctors save prescriptions simultaneously.

Root cause:

The application does not correctly isolate patient information during simultaneous prescription operations.

Possible contributing problems include shared temporary data, incorrect transaction handling or improper association between a prescription and its patient record. The exact technical cause must be confirmed through investigation.

Prevention strategy: Design Inspection and Concurrency Testing

The hospital should introduce database and architecture reviews before implementing critical patient-data operations.

Important preventive activities include:

1. Use unique patient and prescription identifiers.
2. Maintain correct relationships between doctors, patients and prescriptions.
3. Implement appropriate database transaction management.
4. Avoid unsafe sharing of patient-specific temporary data.
5. Test simultaneous prescription creation by multiple doctors.
6. Verify that each generated PDF contains only the correct patient's details.
7. Perform security and data-isolation testing.

Example test case:

Test condition	**Expected result**
Doctor A saves Patient A's prescription.	PDF displays Patient A's details.
Doctor B saves Patient B's prescription simultaneously.	PDF displays Patient B's details.

Both doctors generate PDFs at the same time.	No patient information is mixed.

Cost and benefit:

Design inspections and concurrency testing require additional technical effort and suitable test environments.

However, the benefit is extremely high because this defect affects confidential medical information and can damage patient trust.

C. Incident I8 – Incorrect Refund Test Result

Testing defect

Problem:

The refund test case was marked as passed because the tester compared the output with an incorrect expected value.

Root cause:

The expected result was not properly verified against the approved refund policy.

The testing process did not include independent review of test cases and expected values.

Prevention strategy: Test Oracle Validation

The hospital should introduce a test case review process in which every expected result is checked against an approved requirement or business rule.

Preventive activities include:

1. Document the refund policy clearly.
2. Verify expected refund amounts independently.
3. Review test cases before execution.
4. Use approved calculation examples.

5. Maintain separate actual and expected result fields.
6. Require evidence before marking a test as passed.

Example test case:

Test element	**Value**
Original payment	₹550
Refund condition	Full refund permitted
Expected refund	₹550
Incorrect expected value	₹500
Actual refund	₹500
Correct result	Fail

Cost and benefit:

Test case reviews require limited additional effort and can be conducted by another team member.

The benefit is high because incorrect test results can allow serious application defects to pass unnoticed.

10.3 Comparison of Prevention Strategies

Incident	**Prevention strategy**	**Effort**	**Expected benefit**
I2 – Sunday booking	Formal requirement reviews	Low	Prevents missing business rules.
I6 – Prescription data mixing	Design inspection and concurrency testing	High	Protects patient information and data accuracy.
I8 – Incorrect refund test	Test oracle validation and peer review	Low	Improves testing accuracy.

10.4 Which Strategy Provides the Best Value?

Among the three strategies, formal requirement reviews provide excellent value for MediQueue because they require relatively little effort and can prevent multiple defects before implementation.

The defect data shows that:

- 30 defects originated during requirements.
- 22 defects originated during design.
- Requirements and design together account for 52 defects out of 100.

This demonstrates that early reviews should be a major part of the hospital's quality improvement strategy.

However, the highest patient-safety benefit comes from correcting the design weaknesses responsible for incident I6. Therefore, the hospital should not ignore critical design and data-protection risks while prioritising low-cost reviews.

10.5 Overall Prevention Plan

The following preventive activities should be introduced:

Development stage	**Preventive activity**	**Expected outcome**
Requirements	Requirement reviews and stakeholder approval	Complete and correct requirements.
Design	Architecture and database inspections	Reliable system structure and data handling.
Coding	Coding standards and peer reviews	Fewer programming mistakes.
Testing	Test case reviews and valid test oracles	Reliable test results.
Integration	API and configuration testing	Reliable external service communication.

Release	Regression and acceptance testing	Reduced production defects.

10.6 Conclusion

The analysis of I2, I6 and I8 demonstrates that defects can originate from different stages of software development.

Formal requirement reviews, design inspections and test oracle validation can prevent many of these problems before they reach patients.

Among these strategies, requirement reviews provide excellent value because they are relatively inexpensive and address a major source of defects. However, critical prescription and patient-data defects must receive the highest immediate safety priority.

Therefore, MediQueue should focus on defect prevention rather than relying only on defect correction after production release.

TASK 11 – RECOMMEND AND DEFEND A QUALITY IMPROVEMENT PLAN

CO1 | K5 – Evaluate | 4 Marks

11.1 Introduction

A quality improvement plan is a structured approach used to improve the software development and testing process.

MediQueue experienced 60 production defects within three months of release. The main reasons include the absence of formal reviews, developer-only testing, insufficient test coverage and a lack of structured defect management.

The hospital must introduce a practical quality improvement plan that reduces defects, improves patient safety and supports future software releases.

The plan should also be suitable for the existing team of five developers.

11.2 Main Objectives of the Quality Improvement Plan

The proposed plan has the following objectives:

1. Reduce the number of defects reaching production.
2. Improve requirement clarity and completeness.
3. Introduce independent testing activities.
4. Improve application reliability and performance.
5. Protect patient information and prescription records.
6. Establish a structured defect repository.
7. Reduce the cost of correcting defects.
8. Support regular software releases.

11.3 Recommended Process Changes and Their Priority

The hospital should introduce improvements in the following order.

1. Introduce Formal Requirement Reviews

The first priority is to improve requirement quality.

Every important business rule must be documented and approved by hospital management.

Examples include:

- Hospital working days and holidays.
- Appointment booking restrictions.
- Senior-citizen eligibility.
- Consultation fees and GST rules.
- Refund policies.
- Prescription access permissions.

This will reduce missing and misunderstood requirements.

2. Conduct Design and Architecture Reviews

The second priority is to review the application architecture and database design.

Special attention should be given to:

- Token generation.
- Concurrent appointment booking.
- Prescription data isolation.
- Payment processing.
- Database transaction management.
- Application performance.

This will help prevent serious design-related defects.

3. Appoint a Dedicated Quality Engineer

The hospital should appoint one dedicated quality engineer who is independent of daily feature development.

The quality engineer should:

- Prepare test plans.
- Review test cases.
- Verify expected results.
- Conduct integration and system testing.
- Perform regression testing.
- Track defects.
- Prepare quality reports.

Developers should continue performing unit tests and code reviews.

4. Introduce Automated Testing

The hospital should automate repeated and important test scenarios.

Examples include:

- Appointment booking.
- Token generation.
- Payment calculations.
- Refund processing.
- Patient registration.
- Prescription generation.

Automation will reduce repeated manual effort and support regular releases.

5. Implement a Defect Repository

All defects should be recorded in one centralised repository.

Each record must contain the defect ID, description, severity, priority, origin, root cause, assigned developer and correction status.

This will improve defect tracking and future process improvement.

6. Establish Release Quality Gates

Before every release, the hospital should verify that:

- Critical defects are resolved.
- Important test cases are passed.
- Regression testing is completed.
- Payment and prescription workflows are validated.
- Security checks are completed.
- Release documentation is updated.

A release should not be approved only because a demonstration is successful.

11.4 Proposed Quality Improvement Roadmap

MediQueue Quality Improvement Roadmap

1. Phase 1 – Process Foundation

   Weeks 1–2

   - Requirement reviews.
   - Testing standards.
   - Defect classification.
   - Appointment and payment rule documentation.
2. Phase 2 – Design and Code Quality

   Weeks 3–4

   - Database and architecture reviews.

- Peer code reviews.
- Input validation improvements.
- Concurrency-control improvements.

3. Phase 3 – Testing Implementation

   Weeks 5–6

   - Dedicated quality engineer.
   - Test case preparation.
   - Automated regression testing.
   - Integration and performance testing.

4. Phase 4 – Quality Monitoring

   Weeks 7–8

   - Defect repository implementation.
   - Quality metrics.
   - Release quality gates.
   - Continuous process improvement.

Proposed initial eight-week implementation roadmap.

11.5 Role of Testers, Reviews and Defect Repository

Component	**Responsibility**	**Expected benefit**
Quality Engineer	Independent testing and quality monitoring.	Identifies defects missed by developers.
Requirement Review	Validates business rules.	Reduces requirement-related defects.
Design Review	Checks architecture and database structure.	Prevents design-related failures.
Code Review	Examines programming logic and standards.	Improves code quality.

Test Case Review	Verifies test conditions and expected results.	Prevents incorrect test decisions.
Defect Repository	Tracks and analyses defects.	Supports defect prevention.
Regression Testing	Checks previously working features after changes.	Reduces repeated failures.

11.6 Expected Effect on Defect Numbers and Costs

The original defect data shows:

Detection phase	Defects detected	Cost per defect
Requirements	2	₹500
Design	3	₹1,500
Coding	15	₹3,000
Testing	20	₹8,000
Production	60	₹40,000

The production phase accounts for the largest correction cost.

The hospital should aim to detect more defects during requirements, design and coding rather than allowing them to reach production.

The expected improvements include:

1. Fewer production defects.
2. Lower defect correction costs.
3. Reduced patient complaints.
4. Improved appointment and payment accuracy.
5. Better prescription data protection.
6. Faster identification and correction of problems.

The exact reduction in defect count cannot be guaranteed without collecting results from future releases. The hospital should measure production defect leakage, defect fixing cost, test coverage and reopened defects to evaluate improvement.

11.7 Trade-offs, Risks and Limitations

Although the quality improvement plan provides several benefits, it also has some limitations.

Challenge	Possible impact	Recommended solution
Limited team size	Developers may have less time for feature development.	Introduce lightweight reviews and risk-based testing.
Additional testing effort	Releases may take longer initially.	Automate frequently repeated tests.
Testing tool cost	Commercial tools may increase expenses.	Start with suitable open-source tools.
Learning requirements	Team members may need additional training.	Conduct regular knowledge-sharing sessions.
Automation maintenance	Test scripts may require updates after application changes.	Review and maintain scripts regularly.
Incomplete requirements	Testing may still miss business problems.	Obtain stakeholder approval for requirements.

11.8 Is the Plan Practical for a Team of Five Developers?

Yes. The plan is practical if the hospital uses a lightweight and clearly defined process.

The five developers can continue handling development and unit testing, while one quality engineer coordinates independent testing.

A possible responsibility distribution is:

Team member	Suggested responsibility

Developer 1	Appointment and token management.
Developer 2	Payment and refund modules.
Developer 3	Prescription and patient records.
Developer 4	Notifications and external integrations.
Developer 5	Application architecture, code reviews and technical coordination.
Quality Engineer	Independent testing, defect tracking and release verification.

This is a suggested allocation, not an existing team structure.

Developers should also participate in peer reviews so that quality does not become the responsibility of only one person.

11.9 Measuring the Success of the Quality Improvement Plan

The hospital should measure the following indicators after implementation.

Quality metric	**Purpose**
Production defect count	Measures defects reaching patients.
Defect leakage percentage	Measures how many defects escape earlier testing stages.
Test case pass percentage	Measures testing progress.
Test coverage	Identifies untested requirements and functionality.
Average defect fixing time	Measures correction efficiency.
Reopened defect count	Identifies ineffective corrections.
Application response time	Measures performance.
Critical security defects	Monitors patient-data protection risks.

These measurements should be reviewed after every release.

11.10 Conclusion

MediQueue requires a structured quality improvement plan to reduce production defects and improve application reliability.

The proposed plan includes requirement reviews, design inspections, independent testing, automation, defect management and release quality gates.

The plan is practical for a team of five developers because it distributes quality responsibilities and uses automation to reduce repetitive testing effort.

Therefore, the hospital should adopt this quality improvement plan and continuously evaluate its effectiveness using measurable quality indicators.

TASK 12 – RE-EVALUATE AND ADAPT THE QUALITY PROCESS

CO1 | K5 – Evaluate | 8 Marks

12.1 Introduction

Software quality processes must continuously evolve according to changes in business requirements, technology and user expectations.

Initially, MediQueue planned to release a new version every six months. The hospital now wants to release a new version every two weeks and integrate the application with the national digital health ID system.

The application will also share patient health records between hospitals.

These changes introduce new challenges related to testing frequency, system integration, data security, privacy and interoperability.

Therefore, the original quality improvement plan must be re-evaluated and modified to support the new requirements.

A. IMPACT ANALYSIS

12.2 What Has Changed from the Original Situation?

The new requirements introduce three major changes.

Aspect	Original situation	Changed situation
Release frequency	One release every six months.	One release every two weeks.
System integration	Mainly internal hospital application modules.	Integration with the national digital health ID system.
Patient information	Patient records managed within MediQueue.	Health records shared between hospitals.

Testing requirements	Periodic system and acceptance testing.	Continuous testing and automated regression testing.
Security requirements	Protection of hospital application data.	Additional protection for shared health records and external data exchange.
Quality management	Testing before major releases.	Continuous quality monitoring for every release.

12.3 Which Assumptions in the Original Plan Are Affected?

The original quality improvement plan assumed that:

1. The application would have a relatively long release cycle.
2. Most application functionality would remain within the hospital's system.
3. Testing could be completed before each major release.
4. Patient data would mainly be accessed within MediQueue.
5. Integration testing would focus on the hospital's internal modules.

These assumptions are no longer sufficient.

With two-week releases, testing must be performed continuously. The national digital health ID integration also requires testing of external APIs, authentication, data exchange and error handling.

Furthermore, sharing health records between hospitals increases the importance of privacy, access control and data accuracy.

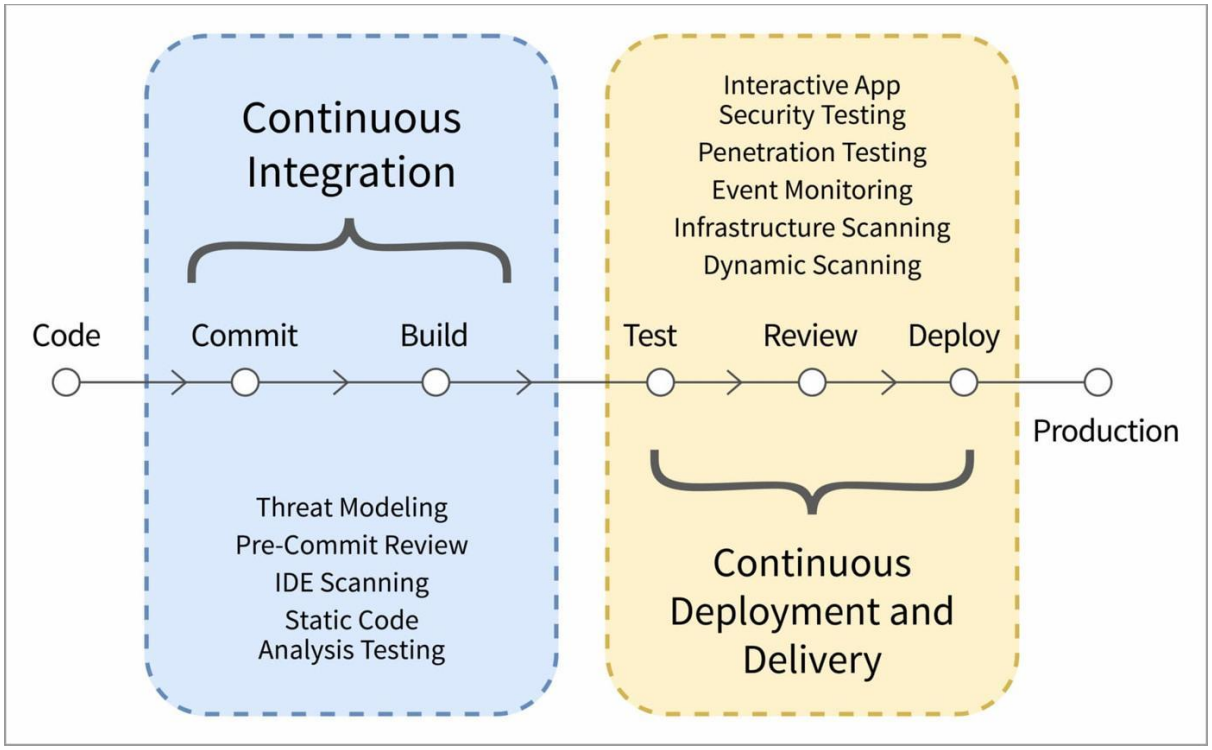
12.4 Why Frequent Releases Change the Role of Testing and the Tester

When software is released every two weeks, the testing process must become faster and more continuous.

The tester can no longer depend only on manual testing immediately before release.

The revised responsibilities include:

- Reviewing requirements during every development cycle.
- Preparing test cases before implementation is completed.
- Performing automated regression testing.
- Testing new features and modified modules.
- Conducting API and integration testing.
- Verifying security and privacy requirements.
- Monitoring production defects after every release.
- Providing release-readiness reports.



Continuous Integration and Continuous Delivery (CI/CD) can help automate repeated testing activities and support faster releases.

12.5 Why Sharing Health Records Increases Defect Severity

Sharing health records between hospitals creates additional risks.

Important risks include:

1. Patient privacy: Unauthorised persons may access confidential medical information.
2. Data accuracy: Incorrect patient information may be transferred between hospitals.
3. Identity mismatch: Health records may become associated with the wrong patient.
4. Authentication problems: Incorrect identity verification may allow unauthorised access.
5. Data transmission security: Information may be exposed during communication between systems.
6. Integration failures: Changes in external services may interrupt record exchange.
7. Data availability: External service failures may prevent hospitals from accessing required records.

For example, if Patient A's prescription is incorrectly associated with Patient B's health ID, another hospital may access inaccurate medical information.

This could affect medical decisions and patient safety.

Therefore, data privacy, identity verification and record accuracy must receive the highest testing priority.

B. EVALUATION OF THE ORIGINAL PLAN

12.6 Do the Original Reviews and Testing Activities Fit into a Two-Week Cycle?

The original quality improvement plan includes requirement reviews, design inspections, code reviews, system testing, regression testing and defect management.

These activities remain necessary, but they must be reorganised to fit a shorter release cycle.

Original activity	**Suitability for two-week releases**	**Required modification**

Requirement reviews	Suitable	Conduct short reviews for every new feature.
Design inspections	Suitable	Review high-risk design changes before implementation.
Code reviews	Suitable	Make peer reviews part of the development workflow.
Manual testing	Partially suitable	Focus manual testing on exploratory and high-risk scenarios.
Automated testing	Highly suitable	Run automated tests whenever code changes.
Regression testing	Highly suitable	Automate important existing test cases.
Performance testing	Suitable	Run regularly and after significant changes.
Security testing	Highly suitable	Include security checks in every relevant release.
Defect repository	Highly suitable	Update continuously throughout development.

The original activities should not be removed. Instead, they should be integrated into a continuous testing process.

12.7 New Defect Origins Introduced by the External Health ID System

Integration with an external national health ID system introduces additional possible defect origins.

New defect origin	**Description**	**Example**
API contract defects	Differences between expected and actual API request or response formats.	MediQueue sends an incorrect patient identification field.
Authentication defects	Problems with identity verification and access permissions.	An expired authentication token is accepted incorrectly.

Data mapping defects	Incorrect mapping of information between two systems.	Patient details are assigned to the wrong fields.
Configuration defects	Incorrect external service settings.	An incorrect API endpoint is configured.
Network defects	Communication failures between systems.	Health records cannot be retrieved due to network interruption.
Security defects	Weak protection during data exchange.	Unauthorised access to shared patient records.
Version compatibility defects	Changes in the external API affect MediQueue.	A new API version changes the response structure.

The hospital must perform integration, compatibility, security and failure-recovery testing to address these risks.

12.8 Is the Existing Defect Repository Still Sufficient?

The original defect repository provides important information such as defect ID, description, severity, priority, origin, root cause and correction status.

However, it should be expanded to manage defects involving external systems.

The following additional fields are recommended:

Additional field	**Purpose**
External System Name	Identifies the connected external service.
API Endpoint	Records the affected API.
API Version	Identifies the version used during testing.
Request and Response Details	Helps investigate integration failures without exposing sensitive patient data.
Authentication Status	Records authentication-related failures.

Data Mapping Information	Identifies incorrect field mapping.
Privacy Impact	Records possible exposure of patient information.
Integration Test Result	Stores the result of external system testing.
External Dependency Status	Records whether the external service was available.

The repository should also maintain audit information about defect corrections and changes affecting patient-data access.

Evaluation

The original defect repository remains useful, but it is not fully sufficient for the expanded application.

It must be modified to include external integration, security, privacy and compatibility information.

C. ADAPTATION OR REDESIGN

12.9 Decision: Modify the Original Quality Process

Recommended decision: Modify

The original quality improvement plan contains useful activities such as requirement reviews, design inspections, independent testing, automation and defect management.

Completely replacing the plan would create unnecessary effort.

However, retaining it without changes would not be sufficient for two-week releases and external health record integration.

Therefore, the hospital should modify the existing process by introducing continuous testing, automated regression testing, API testing and stronger security validation.

12.10 Revised Quality Improvement Process

The modified process should contain the following stages:

Stage 1 – Requirement Analysis

- Review requirements for every new feature.
- Obtain approval from hospital stakeholders.
- Identify privacy and security requirements.
- Document external integration requirements.

Stage 2 – Design Review

- Review application architecture.
- Verify patient identity and data mapping.
- Review database relationships.
- Check external API communication design.

Stage 3 – Development and Code Review

- Follow coding standards.
- Conduct peer code reviews.
- Perform unit testing.
- Use secure coding practices.

Stage 4 – Continuous Automated Testing

- Run automated unit tests.
- Perform integration testing.
- Execute automated regression tests.
- Validate API requests and responses.

Stage 5 – Security and Performance Testing

- Verify authentication and authorisation.
- Test secure data transmission.

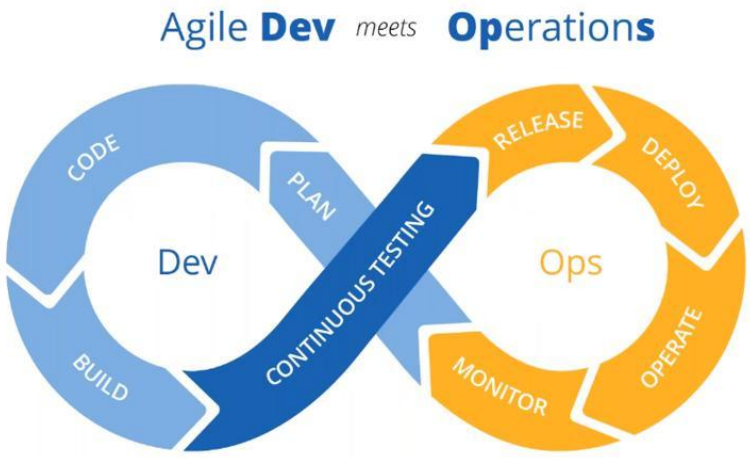
- Check access permissions.
- Perform load and performance testing.

Stage 6 – Release Validation

- Verify all critical test cases.
- Check unresolved defects.
- Perform user acceptance testing for important changes.
- Approve releases only after quality criteria are satisfied.

Stage 7 – Production Monitoring

- Monitor application performance.
- Track production defects.
- Review integration failures.
- Update the defect repository.
- Use feedback to improve the next release.



12.11 Revised Role of the Tester

The tester's role must expand from traditional manual testing to continuous quality engineering.

Previous role	Revised role
Testing before release	Continuous testing throughout development.
Manual test execution	Combination of manual and automated testing.
Internal application testing	Internal and external API testing.
Functional validation	Functional, security, privacy and integration validation.
Defect reporting	Defect analysis and prevention.
Release testing	Continuous release-readiness verification.

The quality engineer should work closely with developers and hospital management to ensure that each release meets the required quality standards.

12.12 Testing Principles That Become More Important

The following testing principles become especially important under the changed conditions.

1. Early Testing

Testing must begin during requirement analysis and design. This helps identify integration and security problems before implementation.

2. Testing Shows the Presence of Defects, Not Their Absence

Even after extensive automated testing, the application cannot be considered completely defect-free. Continuous monitoring is necessary.

3. Exhaustive Testing Is Impossible

The team cannot test every possible patient record, API response and integration condition. Risk-based testing should be used to prioritise critical scenarios.

4. Defect Clustering

Testing should focus on high-risk areas such as prescription management, health ID integration, payment processing and patient records.

5. Pesticide Paradox

Automated test cases must be reviewed and updated regularly to identify new defects introduced by software changes and external API updates.

6. Testing Is Context Dependent

Because MediQueue handles healthcare information, testing must give special attention to patient safety, privacy, data accuracy and security.

12.13 How External System Defects Should Be Classified and Tracked

The hospital should introduce a separate category for external integration defects.

Defect category	**Example**	**Recommended action**
API defects	Incorrect request or response format.	Verify API contracts and data formats.
Authentication defects	Invalid identity verification.	Review authentication and token handling.
Data mapping defects	Incorrect patient information mapping.	Validate field mapping and patient identifiers.
Security defects	Unauthorised health record access.	Conduct security testing and access-control reviews.
Configuration defects	Incorrect API key or endpoint.	Introduce configuration management.
Availability defects	External service is unavailable.	Test error handling and recovery mechanisms.
Compatibility defects	External API version changes.	Perform compatibility and regression testing.

Every external defect should be recorded with its affected system, API version, test evidence, severity, privacy impact and correction status.

The hospital should also distinguish between defects in MediQueue and failures originating from the external service. This helps identify the responsible system and coordinate corrective action.

D. EVIDENCE-BASED DEFENCE

12.14 Justification for Modifying the Original Plan

The decision to modify the original quality improvement plan is supported by the defect data, release frequency and new healthcare integration requirements.

1. Evidence from Defect Data

The original project experienced:

- 30 requirements defects.
- 22 design defects.
- 38 coding defects.
- 20 defects detected during testing.
- 60 defects detected after production release.

The total fixing cost was ₹26,10,500 using the supplied phase-wise detection counts and fixing costs.

Production defects alone accounted for ₹24,00,000.

This demonstrates the need for early defect detection and continuous testing.

2. Evidence from Release Frequency

The release cycle has changed from six months to two weeks.

A manual testing process performed only before release will not be sufficient.

Automated regression testing and continuous integration are required to identify problems quickly.

3. Evidence from External Integration

The national health ID integration introduces new risks involving API compatibility, authentication, patient identity and data exchange.

The original testing process must be expanded to include external integration and security testing.

4. Evidence from Patient Safety and Privacy

Incident I6 already demonstrated that patient information could be mixed between prescriptions.

When records are shared between hospitals, the consequences of incorrect data handling may become more serious.

Therefore, privacy, access control and data accuracy must receive the highest priority.

5. Evidence from Team Size

The project still has only five developers.

Introducing a large and complicated testing department may not be practical.

Instead, one dedicated quality engineer can coordinate testing while developers perform unit tests, code reviews and automated checks.

This approach improves quality without requiring a large increase in team size.

12.15 Comparison of Original and Modified Quality Processes

Quality activity	Original plan	Modified plan
Requirement analysis	Formal requirement reviews	Reviews during every two-week cycle
Design review	Periodic inspections	Risk-based inspections for every significant change
Developer testing	Unit testing	Unit testing with automated execution
Independent testing	Dedicated quality engineer	Dedicated quality engineer with continuous testing responsibilities
Regression testing	Before release	Automated execution throughout development
Integration testing	Internal module testing	In ternal and external API testing
Security testing	General application security	Stronger patient-data privacy and access-control testing
Defect repository	Basic defect tracking	Extended repository with integration and privacy fields
Release approval	Final testing and approval	Defined quality gates for every release
Monitoring	General production monitoring	Continuous defect and integration monitoring

12.16 Expected Benefits of the Modified Plan

The revised quality process is expected to provide the following benefits:

1. Reduced production defect leakage.
2. Faster identification of software problems.
3. Improved reliability of two-week releases.
4. Better patient-data accuracy.
5. Improved security and privacy protection.
6. More reliable communication with the external health ID system.
7. Reduced repeated testing effort through automation.
8. Better defect tracking and root-cause analysis.

9. Improved patient and hospital staff satisfaction.
10. More consistent software quality across releases.

12.17 Conclusion

The new requirements significantly change the quality needs of the MediQueue application. Releasing software every two weeks requires continuous testing and automation, while integration with the national digital health ID system introduces additional security, privacy and interoperability risks.

The original quality improvement plan is a good foundation, but it cannot be retained without modification.

Therefore, the hospital should modify the existing process by introducing continuous integration, automated regression testing, external API validation, security testing and an expanded defect repository.

This modified approach is practical for a small development team and provides a systematic method for maintaining software quality while supporting frequent releases and secure health record sharing.

MARK DISTRIBUTION

Task	Marks
Task 1 – Evaluate Testing as an Engineering Activity	5
Task 2 – Evaluate the Role of Process in Software Quality	2
Task 3 – Evaluate Testing as a Process	6
Task 4 – Apply and Critique Basic Definitions	4
Task 5 – Evaluate the Application of Testing Principles	3
Task 6 – Evaluate the Tester's Role	3
Task 7 – Trace the Origins of Defects	4
Task 8 – Evaluate Defect Classes and Severity	4
Task 9 – Design and Appraise a Defect Repository	3
Task 10 – Evaluate Defect Examples and Prevention Strategies	4
Task 11 – Recommend and Defend a Quality Improvement Plan	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50